ECS

19/4/26 14:32

Amazon Elastic Container Service Cheat Sheet

- **Amazon Elastic Container Service (Amazon ECS)** is a fully managed container orchestration service that makes it easy to deploy, manage, and scale containerized applications.
- It creates a consistent deployment and build experience, allowing you to manage and scale batch and ETL workloads, and build sophisticated application architectures on a microservices model.
- Amazon ECS is a regional service.

Features

- You can create ECS clusters within a new or existing VPC.
- After a cluster is up and running, you can define task definitions and services that specify which Docker container images to run across your clusters.
- AWS Compute SLA guarantees a Monthly Uptime Percentage of at least 99.99% for Amazon ECS.
- **Amazon ECS Exec** is a way for customers to execute commands in a container running on Amazon EC2 instances or AWS Fargate. ECS Exec gives you interactive shell or single command access to a running container.
- **Amazon ECS Anywhere** allows you to run ECS tasks on your own on-premises infrastructure (VMs or bare metal) while managing them through the standard AWS ECS console and APIs.
- **Amazon ECS Service Connect** provides managed service-to-service communication that simplifies network configuration and increases resilience without needing complex sidecar proxies.
- Task scale-in protection allows you to protect specific critical tasks, such as long-running batch jobs, from being terminated during auto-scaling scale-in events.

Components

- Containers and Images
    - Your application components must be architected to run in **containers** — containing everything that your software application needs to run: code, runtime, system tools, system libraries, etc.
    - Containers are created from a read-only template called an **image**.
    - Images are typically built from a **Dockerfile**, a plain text file that specifies all of the components that are included in the container. These images are then stored in a **registry** from which they can be downloaded and run on your cluster.
    - When you launch a container instance, you have the option of passing *user data* to the instance. The data can be used to perform common automated configuration tasks and even run scripts when the instance boots.
    - Docker Volumes can be a local instance store volume, EBS volume, or EFS volume. Connect your Docker containers to these volumes using Docker drivers and plugins.
- Task Components
    - **Task definitions** specify various parameters for your application. It is a text file, in JSON format, that describes one or more containers, up to a maximum of ten, that form your application.
    - Task definitions are split into separate parts:
        - Task family – the name of the task, and each family can have multiple revisions.
        - IAM task role – specifies the permissions that containers in the task should have.
        - Network mode – determines how the networking is configured for your containers.
        - Container definitions – specify which image to use, how much CPU and memory the container is allocated, and many more options.
        - Volumes – allow you to share data between containers and even persist the data on the container instance when the containers are no longer running.
        - Task placement constraints – lets you customize how your tasks are placed within the infrastructure.
        - Launch types – determines which infrastructure your tasks use.
- Tasks and Scheduling
    - A **task** is the instantiation of a task definition within a cluster. After you have created a task definition for your application, you can specify the number of tasks that will run on your cluster.
        - Each task that uses the Fargate launch type has its own isolation boundary and does not share the underlying kernel, CPU resources, memory resources, or elastic network interface with another task.
    - The **task scheduler** is responsible for placing tasks within your cluster. There are several different scheduling options available.
        - REPLICA — places and maintains the desired number of tasks across your cluster. By default, the service scheduler spreads tasks across Availability Zones. You can use task placement strategies and constraints to customize task placement decisions.
        - DAEMON — deploys exactly one task on each active container instance that meets all of the task placement constraints that you specify in your cluster. When using this strategy, there is no need to specify a desired number of tasks, a task placement strategy, or use Service Auto Scaling policies.
    - You can upload a new version of your application task definition, and the ECS scheduler automatically starts new containers using the updated image and stop containers running the previous version.
    - Amazon ECS tasks running on both Amazon EC2 and AWS Fargate can mount Amazon Elastic File System (EFS) file systems.
    - You can enable scale-in protection for specific tasks to prevent them from being terminated by Service Auto Scaling. This is critical for long-running processes (such as queue-processing jobs or gaming sessions) that must complete before the instance is terminated.
- Clusters
    - When you run tasks using ECS, you place them in a **cluster**, which is a logical grouping of resources.
    - Clusters are Region-specific.
    - **Launch Types:** Clusters can contain tasks using the **Fargate**, **EC2**, and **External** (ECS Anywhere) launch types simultaneously.
        - **Fargate:** ECS manages your cluster resources (serverless).
        - **EC2:** Your clusters comprise container instances that you manage. Each container instance may belong to only one cluster at a time.
    - **Capacity Providers (Cluster Auto-Scaling):**
        - Capacity Providers are the modern interface for managing the infrastructure backing your cluster.
        - They allow ECS to automatically scale your **EC2 Auto Scaling Groups** (ASG) in and out based on the exact capacity needed by your pending tasks (Managed Scaling).
        - You can create a **Capacity Provider Strategy** to split tasks across different providers (e.g., spreading tasks between On-Demand and Spot instances).
    - Before you can delete a cluster, you must delete the services and deregister the container instances inside that cluster.
    - **Amazon ECS Anywhere:**
        - Enables you to run ECS tasks on your own on-premises infrastructure (VMs, bare metal, or other clouds) by registering them to your ECS cluster.
        - **External Launch Type:** On-premises servers are registered as “External” instances. You manage them using the same ECS console, APIs, and task definitions as you do for your cloud workloads.
        - Requires the **Amazon SSM Agent** and **ECS Container Agent** to be installed on your on-premises servers.
        - External instances need outbound connectivity to AWS APIs (via internet or AWS Direct Connect/VPN) to communicate with the ECS control plane.
- Services
    - ECS allows you to run and maintain a specified number of instances of a task definition simultaneously in a cluster.
    - In addition to maintaining the desired count of tasks in your service, you can optionally run your service behind a load balancer.
    - There are two deployment strategies in ECS:
        - **Rolling Update**
            - This involves the service scheduler replacing the currently running version of the container with the latest version.
            - The number of tasks ECS adds or removes from the service during a rolling update is controlled by the deployment configuration, which consists of the minimum and maximum number of tasks allowed during service deployment.
        - **Blue/Green Deployment with AWS CodeDeploy**
            - This deployment type allows you to verify a new deployment of a service before sending production traffic to it.
            - The service must be configured to use either an Application Load Balancer or Network Load Balancer.
- Container Agent
    - The **container agent** runs on each infrastructure resource within an ECS cluster.
    - It sends information about the resource’s current running tasks and resource utilization to ECS, and starts and stops tasks whenever it receives a request from ECS.
    - The container agent is only supported on Amazon EC2 instances.
- Service Load Balancing
    - Amazon ECS services support the Application Load Balancer, Network Load Balancer, and Classic Load Balancer ELBs. Application Load Balancers are used to route HTTP/HTTPS (or layer 7) traffic. Network Load Balancers are used to route TCP or UDP (or layer 4) traffic. Classic Load Balancers are used to route TCP traffic.
    - You can attach multiple target groups to your Amazon ECS services that are running on either Amazon EC2 or AWS Fargate. This allows you to maintain a single ECS service that can serve traffic from both internal and external load balancers and support multiple path-based routing rules and applications that need to expose more than one port.
    - The Classic Load Balancer doesn’t allow you to run multiple copies of a task on the same instance. You must statically map port numbers on a container instance. However, an Application Load Balancer uses **dynamic port mapping**, so you can run multiple tasks from a single service on the same container instance.
    - If a service’s task fails the load balancer health check criteria, the task is stopped and restarted. This process continues until your service reaches the number of desired running tasks.
    - Services with tasks that use the *awsvpc* network mode, such as those with the Fargate launch type, do not support Classic Load Balancers. You must use NLB instead of TCP.
- Amazon ECS Service Content
    - Amazon ECS Service Connect is a managed networking capability that simplifies service-to-service communication within ECS.
    - It provides a way to manage service discovery, connectivity, and traffic observability without configuring or maintaining external load balancers or a complex service mesh (such as App Mesh) for internal traffic.
    - **Simplified Service Discovery:** Automatically maps logical names (e.g., frontend or db) to the endpoints of your services, allowing them to talk to each other easily within the cluster.
    - **Resilience:** Handles connection draining, automatic retries, and health checks to ensure reliable communication between services.
    - **Traffic Observability:** Provides rich telemetry data (request counts, latency, error rates) directly in the ECS console and in Amazon CloudWatch, giving you deep visibility into how your services interact.

AWS Fargate

- You can use Fargate with ECS to run containers without having to manage servers or clusters of EC2 instances.
- You no longer have to provision, configure, or scale clusters of virtual machines to run containers.
- Fargate only supports container images hosted on Elastic Container Registry (ECR) or Docker Hub.

Task Definitions for Fargate Launch Type

- Fargate task definitions require that the network mode is set to *awsvpc*. The *awsvpc* network mode provides each task with its own elastic network interface.
- Fargate task definitions require that you specify CPU and memory at the task level.
- Fargate task definitions only support the *awslogs* log driver for the log configuration. This configures your Fargate tasks to send log information to Amazon CloudWatch Logs.
- Task storage is **ephemeral**. After a Fargate task stops, the storage is deleted.
    - By default, Fargate tasks receive 20 GiB of ephemeral storage. You can now configure this up to 200 GiB for workloads that need more scratch space for processing data.
- Fargate supports AWS Graviton2 (ARM64) processors, which can provide up to 40% better price/performance compared to x86-based processors for compatible applications.
- Amazon ECS tasks running on both Amazon EC2 and AWS Fargate can mount Amazon Elastic File System (EFS) file systems.
- Put multiple containers in the same task definition if:
    - Containers share a common lifecycle.
    - Containers are required to be run on the same underlying host.
    - You want your containers to share resources.
    - Your containers share data volumes.
- Otherwise, define your containers in separate task definitions so that you can scale, provision, and de-provision them separately.

Task Definitions for EC2 Launch Type

- Create task definitions that group the containers that are used for a common purpose, and separate the different components into multiple task definitions.
- After you have your task definitions, you can create services from them to maintain the availability of your desired tasks.
- For EC2 tasks, the following are the types of data volumes that can be used:
    - **Docker volumes**
    - **Bind mounts**
- Private repositories are only supported by the EC2 Launch Type.

Amazon Elastic Container Service Monitoring

- You can configure your container instances to send log information to CloudWatch Logs. This enables you to view different logs from your container instances in one convenient location.
- With CloudWatch Alarms, watch a single metric over a time period that you specify, and perform one or more actions based on the value of the metric relative to a given threshold over a number of time periods.
- Share log files between accounts, and monitor CloudTrail log files in real time by sending them to CloudWatch Logs.

Amazon Elastic Container Service Tagging

- ECS resources, including task definitions, clusters, tasks, services, and container instances, are assigned an Amazon Resource Name (ARN) and a unique resource identifier (ID). These resources can be tagged with values that you define, to help you organize and identify them.

Amazon Elastic Container Service Pricing

- With Fargate, you pay for the amount of vCPU and memory resources that your containerized application requests. vCPU and memory resources are calculated from the time your container images are pulled until the Amazon ECS Task terminates.
- There is no additional charge for the EC2 launch type. You pay for AWS resources (e.g. EC2 instances or EBS volumes) you create to store and run your application.
- For Amazon ECS Anywhere Pricing, you pay a flat hourly fee for each managed on-premises instance registered to your ECS cluster (approximately $0.01025 per instance per hour). You do not pay for the on-premises compute resources themselves (since you own them), only for the ECS management capability.

From <https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>